

Unit -IV

Chapter-5

5.3 Assembly language programming

5.3 Assembly language programming

- Now we can start assembly programming using the Keil Microvision V assembler.
- The step-by-step method of using this IDE for Assembly Language

5.3.1 Assembler Directives

For each assembler, there are 'directives'.

They are specific for an assembler and are non-executable statements, which means that they don't cause any 'execution' by the processor.

Directives are related solely to the assembler.

We need a few directives of the Keil Assembler to get started in assembly programming.

5.3.1.1 AREA

The “**AREA**” directive instructs the assembler to assemble a new code or data section".

The above is a statement given in the user manual of Keil Microvision.

It implies that the assembler understands two types of information —code and data.

Data is usually stored in R/W memory (RAM) while code is in Read only memory (ROM).

The assembler requires the names of these areas of code or data.

- Typical statements that use the AREA directive are

AREA LINEAR, CODE, Read only

AREA NOTE, DATA, R/W

- It is not mandatory to use the words 'Read Only' and 'R/W' because by default, data is in R/W memory and code is in Read Only Memory.
- Code is always in ROM, though data is also allowed to be stored there.
- As a matter of fact, the R/W memory is volatile and is used only for intermediate storage of data in the course of instruction execution

5.3.1.2 ENTRY

The ENTRY directive marks the first instruction to be executed within an application.

Because an application cannot have more than one entry point, the ENTRY directive can appear in only one of the source modules.

5.3.1.3 END

This directive is the last line in an assembly module and it indicates that the assembler need not read beyondthat line.

5.3.1.4 DCB, DCW and DCD

The ARM processor has a word length of 32 bits, but it can handle data of 8 bits (byte) and 16 bits (half word) also.

The Keil assembler, however, uses the notations of byte, word and double word for 8 bits, 16 bits and 32 bits, respectively.

The directives used are DCB for data byte, DCW for data word and DCD for data double word.

The following lines show how these directives are used.

These directives are used by the assembler to store data in ROM.

Example

BAG DCB 0x32, 56, 0x7E

BAG DCW 0x2234, 0x ED37, 0x9067

BOG DCD 0x DE345678, 0x34009812

The first line says that three bytes are to be stored in contiguous locations with labels, RAG, RAG+1, and RAG+2. Recollect that each address always corresponds to one byte of data. The second line indicates that 16 bit data is being stored in locations BAG, BAG+2 and BAG+4. The third line states that 32 bit data is to be stored in addresses with labels BOG and BOG+4.

5.3.1.5 equ

The EQU directive is for equating a label to a constant.

When the label is encountered by the assembler during the process of assembly, it is replaced by its actual value.

Examples of the usage of this directive are

- **RAPID EQU 65**
- **STRT_ADDR EQU 0x84000000**

Write an ALP (assembly language program) to add the first 5 natural numbers. The sum must be in a register.

AREA NAT, CODE, READONLY

ENTRY

MOV R0,#0 ;R0=0

MOV R1,#0 ;R1=0

BACKK ADD R0,R0,#1 ;Increment R0 by 1

ADD R1,R1,R0 ;Add the numbers.The sum is in R1

CMP R0,#5 ;check if R0=5

BNE BACKK ;repeat if R0 is not equal to 5

GO B GO ;continue branching to GO

END ;end of the assembly module

Write and test a program for adding the first ten odd numbers (starting from 1)

```
                AREA ODD, CODE, READONLY
ENTRY
    MOV R1,#1    ;R1=1
    MOV R2,#9;R2=9, the counter
    MOV R3,#1    ;R3=1
BACKK ADD R3,R3,#2    ;R3 has the odd numbers
      ADD R1,R1,R3    ;R1 contains the final sum
      SUBS R2,R2,#1    ;R2 is the counter for 10 numbers
      BNE BACKK ;repeat the addition until R2=0
GO      B GO
END
```

5.4 Accessing memory

We already know that ARM is a load-store architecture.

This means that only the load and store instructions access memory directly.

There are load/store instructions with single registers, as well as with multiple registers.

5.4.1 Single register data transfer

- The word 'load' indicates that data from memory is to be moved to a register. 'Store' means that data from a register is saved in memory
- The format for a load/store instruction is
LDR/STR {<cond>}<Rd>, <addressing mode>
- The operation of load/ store has a register Rd in which the data is placed. The 'addressing mode' implies that an effective address is generated by address calculation.

Example

LDR R2, [R7] ; copy into R2, the content of memory, as pointed by R7

STR R2, [R7] ; store the content of R2, into the address pointed by R7

The address of the memory location for the load/store operation is placed in a register, called the base register. The address may be generated and put into the base register in various calculation formats using shifting, adding offsets, summing the contents of more registers etc.

Find the effective address of memory as specified by the following instructions

1. LDR R4, [R7, ROR #4]
2. STR R7, [R3, R4, LSL #2]
3. LDR R6, [R2, #4]
4. STR R8, [R1, R2, ASR #6]
5. LDR R1, [R7, R6]

- The effective address is $R7 = R7$ rotated right 4 times.
- The effective address is $R3 = R3 + R4$ rotated left logically 2 times.
- The effective address is $R2 = R2 + 4$
- The effective address is $R1 + R2$ right shifted (arithmetically) 6 times
- The effective address is $R7 = R7 + R6$

5.4.2 Loading/Storing data of different lengths

- Data to be loaded or stored may be in bytes, half words (16 bits) or words (32 bits).
- Data may be signed or unsigned, as well.
- To make this explicit, the load and store instructions may be appended with the characters, B for byte, H for half word and S for signed.

Load/Store instructions

Instruction	Meaning
LDR	Load Word
LDRH	Load Half Word
LDRSH	Load Signed Half Word
LDRB	Load Byte
LDRSB	Load Signed Byte
STR	Store Word
STRH	Store Half Word
STRB	Store Byte

How is data accessed from memory with the following instructions, given that R7=0x00004400 and that memory contents are as given in Table (assume the little endian scheme)?

1. **LDR R3, [R7]**
2. **LDRB R3,[R7]**
3. **LDRH R3, [R7]**
4. **LDRSH R3, [R7]**

Address	Data
0x00004400	89
0x00004401	E6
0x00004402	F3
0x00004403	94

LDR R3, [R7]

This instruction pertains to a word (32 bits) of data to be loaded into the destination register. After the execution of the above instruction, R3=0x94F3E689.

LDRB R3, [R7]

After the execution of the above instruction, R3=0x00000089. This is because only a byte of the data will be taken from memory and so the lowest byte of the word (in the lowest address) pointed by R7, is loaded to R3.

LDRH R3, [R7]

After the execution of the above instruction, R3=0x0000E689. This is because the half word of the data will be taken from memory and the lower 16 bits of the word pointed by R7, is loaded to R3.

LDRSH R3, [R7]

After the execution of the above instruction, R3=0xFFFFE689. The lower half word of the data is 0xE689. The instruction LDRSH considers this to be a 'signed halfword' and when loading it to a 32-bit register, the sign bit is extended to the upper 16 bits.

The content of registers, R6 and R3, and memory are as given below. R6= 0x40002200, R3=0x3498CE56

Show the content of memory after the execution of the following instructions

- STRB R3, [R6]**
- STRH R3, [R6]**
- STR R3, [R6]**



Address	Data
0x40002200	00
0x40002201	00
0x40002202	00
0x40002203	00

- **STRB R3, [R6]**

Address	Data
0x40002200	56
0x40002201	00
0x40002202	00
0x40002203	00

- **STRH R3, [R6]**

Address	Data
0x40002200	56
0x40002201	CE
0x40002202	00
0x40002203	00

- **STR R3, [R6]**

Address	Data
0x40002200	56
0x40002201	CE
0x40002202	98
0x40002203	34

5.4.3 Pre Indexing, Post Indexing and Write Back

- Consider the Instruction

STR R0, [R7,#2] and LDR R0,[R7,ROR #4]

- In the first instruction, it calculates the effective address as $R7 = R7 + 2$.
- In the second instruction, it calculates the address as $R7 = R7 \text{ ROR } \#4$.

What is the content of R7 after the execution of previous instruction?

- The content of R7 remains the same as it was before the instruction execution.
- Means the address calculation before, no change in R7

Write Back(!):

- If it necessary to save the effective address into the base registers,
- Instruction should have the concept of “ Write Back”.
- Means the instruction need to have with the sign “ !”.
- Write back instruction is:

STR R0, [R7,#2]! and LDR R0,[R7,ROR #4]!

Examples

What is the content of the base register R6 after the execution of each of the following instructions, given that R6=0x44001100 and R2 = 0x02.

1. LDR R2,[R6,#0x0056]!
2. STR R1,[R6,R2]!
3. LDR R3,[R6,LSL #4)

1 . LDR R2,[R6,#0x0056]!

The effective address is calculated as $R6 = R6 + 0x0056 = 0x44001156$, because of write back, this number will be saved in R6 after instruction execution.

2. STR R1,[R6,R2]!

The effective address is calculated as $R6 = R6 + R2 = 0x44001102$ because of write back, this number will be saved in R6 after instruction execution.

3. LDR R3,[R6,LSL #4]

The effective address is calculated as $R6 = R6, \text{LSL}\# 4$. Because the instruction does not have the write back feature (!), after instruction execution, R6 will retain its previous content of 0x44001100.

5.4.4 Post Indexing

- In this mode, the effective address calculation is done after the execution of the specific instruction.
- Thus, write back is inherently implied.
- Take the case of the instruction `LDR R2, [R7], #2`
- Here the data pointed by the content of R7 is first copied to R2.
- After that, the content of R7 (which is the base register) is changed to $R7 + 2$.

5.4.5 Immediate mode of addressing

- This is the mode when an immediate data is used as an operand.
- Consider the instruction

`MOV R1, #10` ; move data 10 to R1

`ADD R4, R2, #3` ; Add 3 to the content of R2, and save the sum in R4

These instructions seems to be ok. But there is an unseen issues involved

What is the biggest constant that can be used in the immediate mode?

- All instructions of ARM are to have same size 32 bits
- If an immediate data is to be included, it must fit in the 32-Bit size allocated for a data processing instruction.
- Means the larger sizes of constants cannot be used in immediate data.

- The figure 5.10 shows the format of a typical data processing instructions of ARM.
- If bit 25 is '0', it is understood that the operand2 is a register.
- If bit 25 is '1', the operand2 is immediate data.
- 12bit space is allocated for operand2.



Figure 5.10 Format of an immediate mode instruction

- The 12bit space is arranged as a four bit rotate value and an eight bit immediate value as shown in figure 5.11.
- The rotation bit field is only 4 bits .
- Internally this rotation field is multiplied by 2, and thus becomes the case of an 8 bit number multiplied an even number of times to get a new number.

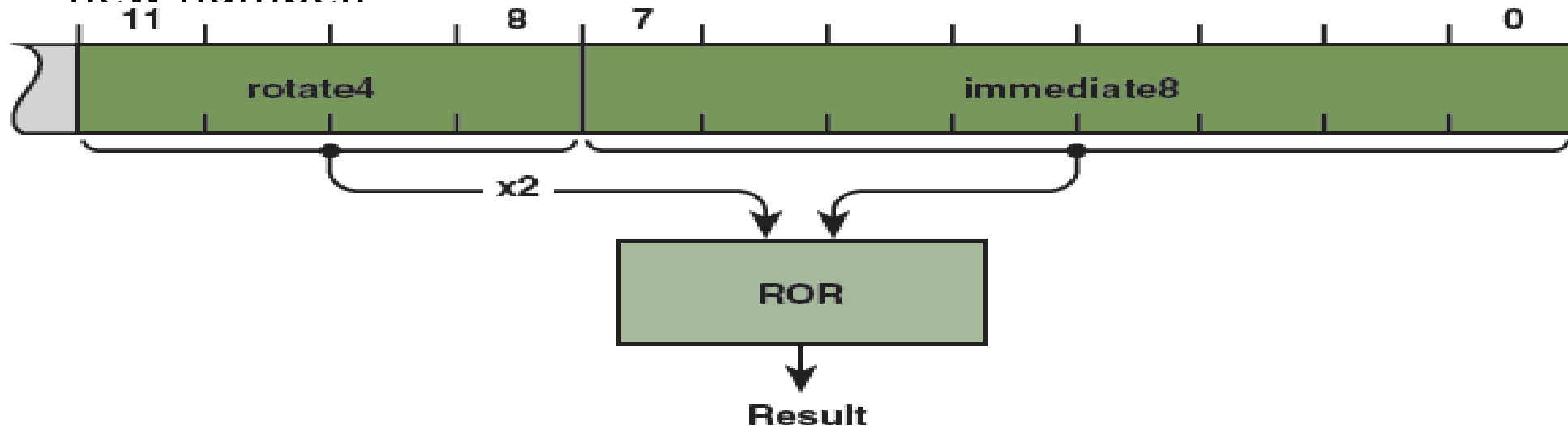


Figure 5.11 Rotate operation for handling immediate data

Perform the ARM rotation scheme for the number 0xE0.

The number in 32 bit space is written as

0000 0000 0000 0000 0000 0000 1110 0000

The number rotated 4 times to the right becomes

0000 0000 0000 0000 0000 0000 0000 1110 i.e. 0x0E

The number rotated 6 times to the right becomes

1000 0000 0000 0000 0000 0000 0000 0011 i.e. 0x80000003

Table 5.12 Constants generated by rotation schemes (Courtesy: ARM ASSEMBLY LANGUAGE: William Hohl)

Decimal values	Equivalent hexadecimal	Step between values	Rotate
0 – 255	0 – 0xff	1	No rotate
256,260,264, ...,1020	0x100 – 0x3fc	4	Right by 30 bits
1024,1040,1056, ..., 4080	0x400 – 0xff0	16	Right by 28 bits
4096,4160,4224, ...,16320	0x1000 – 0x3fc0	64	Right by 26 bits

Find the constants obtained by rotation, for the following cases.

1. Rotate 0x30, to the right 28 times
2. Rotate 0x96, to the right 14 times
3. Rotate 0x5C, to the right 10 times
4. Rotate 0x09, to the right 15 times
5. Rotate 0xFC, to the right 6 times

SOLUTION

1. 0000 0000 0000 0000 0000 0000 0011 0000

Rotating 28 times to the right (which is the same as rotating 4 times to the left) is:

$$0000\ 0000\ 0000\ 0000\ 0000\ 0011\ 0000\ 0000 = 0x00000300$$

2. 0000 0000 0000 0000 0000 0000 1001 0110

Rotating it to the right 14 times will make it

$$0000\ 0010\ 0101\ 1000\ 0000\ 0000\ 0000\ 0000 = 0x02580000$$

3. The rotation generates the number 0x17000000
4. The rotation generates the number 0x00120000
5. The rotation generates the number 0xF0000003

- When we want larger constant, it is not possible to write it in an instruction like MOV R1, #n, where n is constant
- Need to use the immediate mode in such a way that the constant that we need is obtained by the ARM rotation scheme
- Not possible by programmer to find out exact 8-bit number which will produce a bigger size constant
- So Assembler will come to an assistant here.
- for that, a pseudo instruction is provided
- The format is LDR Rd, =constant ; where Rd is destination register

ex: LDR R1, =0x44444444

- But all constants are not obtainable by such a scheme.
- Consider 0x77777777.
- This constant cannot be generated in this way.
- In such a case, the assembler is used to store the required constant in memory and take it from memory using the LDR instruction.
- The area in memory where such constants are loaded is called a “literal pool”.
- programmer need only write the pseudo instruction.
- The assembler wil handle the matter in an efficient manner.
- The sequence of actions that follow is highlighted below.
- Take the case of needing a constant 0x77777777.
- The pseudo instruction we write is

LDR R1 ,=0x77777777

AREA IMMID,CODE,READONLY

ENTRY

LDR R3, = 0x6D0000

LDR R4, = 0X77777777

ADD R5,R3,R4

GO B GOEND

Space for literal pool is:

- After the END directive, so that it can be stored in ROM in the space after the code.
- If the program is large and the pseudo instruction for it is far away from the end of the program, the literal pool may be beyond the range of +/-4KB.
- In such cases a directive LTORG(Origin of literal pool) is used.
- This directive may be included in the program to specify where we want the literal pool to be.

```
                AREA IMMID,CODE,READONLY
ENTRY
                LDR R3, = 0x6D0000
                LDR R4, = 0x77777777
                ADD R5,R3,R4
LTORG
                .....
                .....
                .....
                .....
                .....
GO B GO
END
```

Placing LTORG in this place gives a warning message 'Literal pool in executable area'.

```

                AREA IMMID,CODE,READONLY
ENTRY
                LDR R3, = 0x6D0000
                LDR R4, = 0x77777777
                ADD R5,R3,R4
                B  LUN
                LTORG
LUN             .....
                .....
                .....
                .....
                .....
                .....
                .....
GO              B GO
END.

```

- Literal pool are to be place in location in such a way that the processor does not consider them as instructions.
- They are best placed after unconditional branch instructions.

5.4.6 Read Only and Read/Write Memory

- We know that ROM is for code and RAM is for temporary storage of data.
- It is important to realize that data in RAM is accessed using instructions of the processor
- while data is placed in ROM using assembler directives like DCD, DCB etc.
- From ROM data can be brought to RAM by first loading it to registers using Load (LDR) instructions
- Later store them using STR instructions

Block data transfer using multiple registers:

- This ISA allows multiple registers to be loaded from contiguous memory locations, with the use of just one instructions.
- Similarly ,the content of Multiple registers may be saved into consecutive locations in memory. Such instructions are referred to as multiple load/store instructions.

LDM {cond}address-mode Rn{!},reg-list{^}

- Cond: is an optional condition
- address-mode: specifies how the memory is sequenced.
- Rn: is the base register for the load operation.
- ! : Specifies base register write back i.e the address in the base register is updated after the transfer.(if the base register is in the register list then no need to specify write back)
- Reg-list: List of symbolic register names and register ranges enclosed in braces. There must be at least one register in the list.
- ^ : This is not to be used in the user/system mode. It is meant for use in the privileged mode.

Addressing modes of the multiple load/store instructions:

- These modes indicate how to change the memory address after each word transfer.
- The addressing modes are:
 - IA:Increment After
 - IB :Increment before
 - DA:Decrement After
 - DB:Decrement Before

Instructions are

LDMIA R7! {R0-R5}

5.4.7.2 LDMIA R7! {R0-R5}

Let us look at a sample Load instruction

The operation will proceed as follows:

1. R7 is the base address, which means that it contains the address of memory to be accessed.
2. First the content of the address pointed by R7 is loaded to R0.
3. Since 'IA', that is 'Increment after', is the addressing mode, the content of R7 is incremented by 4. **This happens mandatorily, as only word (32 bit) transfers are supported by LDM and STM instructions.**
4. The content of the address pointed by the new value of R7 is loaded to R1.
5. The content of R7 is incremented again, and loading to R2,R3, R4 and R5 occurs.
6. Because of the write back feature (!), at the end of the operation, the value in R7 is the final updated address.

Note: Consider the instruction **LDmia R7!,{R10,R1-R2}**. It might seem that the first data is to be loaded in R10, but that is not correct. Whatever be the order in which the register list is written, the processor operation stipulates that data from the lowest address will always be loaded to the lowest register in the list, that is, R1 in this case.

If the instruction is **LDMDa R7!,{R10,R1-R2}**, the same logic has to be applied. R10 will be loaded with the data in the initial address in R7, because that is the highest address. As the address decrements after each operation, the registers R2 and then R1 is loaded.

The STM instruction:

Syntax: STM{cond} address-mode Rn!, reg-list{^}

- The operation is just the reverse of the LDM instruction.
- Here the contents of the registers are stored in memory location indexed by the base register.

STMDB R1!,{R7,R2-R4} if R1=0X44400088

- The base register is R1, it contains the number 0X44400088. Since the DB suffix is used, the R1 register is decremented by 4 and it contains the number 0X44400084.
- The data in R7 is saved to this location.
- The content of R1 is decremented by 4 and the data in R4 is saved in this address.
- These operations of decrementing the base register is repeated for storing the data in R3 and R2.
- At the end of the execution of the instruction, the value in R1 is the final updated value because write back is stipulated.

5.4.7.4 Realization of stacks

In this ISA, there is no PUSH and POP operation that we normally associate with stacks. Instead, the LDM and STM instructions are used for POP and PUSH operations, respectively. We know that with LDM, the content of a number of locations in memory can be popped into registers. Similarly, STM suffices for the pushing of the contents of registers into memory.

We know that a stack is a data structure which we can define as we wish. There can be decending/ascending stacks which may be full or empty as the following definitions indicate.

1. **Descending or ascending:** The stack grows downwards, starting with a high address and progressing to a lower one (a descending stack), or upwards, starting from a low address and progressing to a higher address (an ascending stack).
2. **Full or empty:** The stack pointer can either point to the last item in the stack (a full stack), or the next free space on the stack (an empty stack).

Table 5.13 illustrates the way these stacks are realized.

The Procedure Call Standard for the ARM Architecture (AAPCS) always uses a full descending stack.

When using a stack, the base register is SP. Writeback is mandatory. The following instructions define push and pop for such a stack (Full Descending):

STMFD SP!,{R0–R5} ;Push onto a full descending stack
STMDB SP!,{R0–R5} ;‘Decrement Before’ is also for pushing to a full descending stack
LDMFD SP!,(R0–R5) ;Pop from a full descending stack

Table 5.13 Types of stacks as addressing modes to be used for them

Stack type	Push	Pop
Full Descending	STMFD(DB)	LDMFD(IA)
Full Ascending	STMFA(IB)	LDMFA(DA)
Empty Descending	STMED(DA)	LDMED(IB)
Empty Ascending	STMEA(IA)	LDMEA(DB)

LDMIA SP!,{R0–R5} ;Increment after is also for popping from a full descending stack.
 Before we use a stack, the stack pointer has to be initialized to the top of the stack to the topmost location in memory that we decide our stack to be.

What does this program do?

```
AREA    FDSTCK,CODE,READONLY
```

```
ENTRY
```

```
    LDR  SP,=0X40000030
```

```
    MOV  R0,#5
```

```
    MOV  R1,#6
```

```
    MOV  R2,#7
```

```
    STMDB SP!, {R0-R2}
```

```
    LDMIA SP!, {R9,R4-R5}
```

```
GO    B      GO
```

```
    END
```

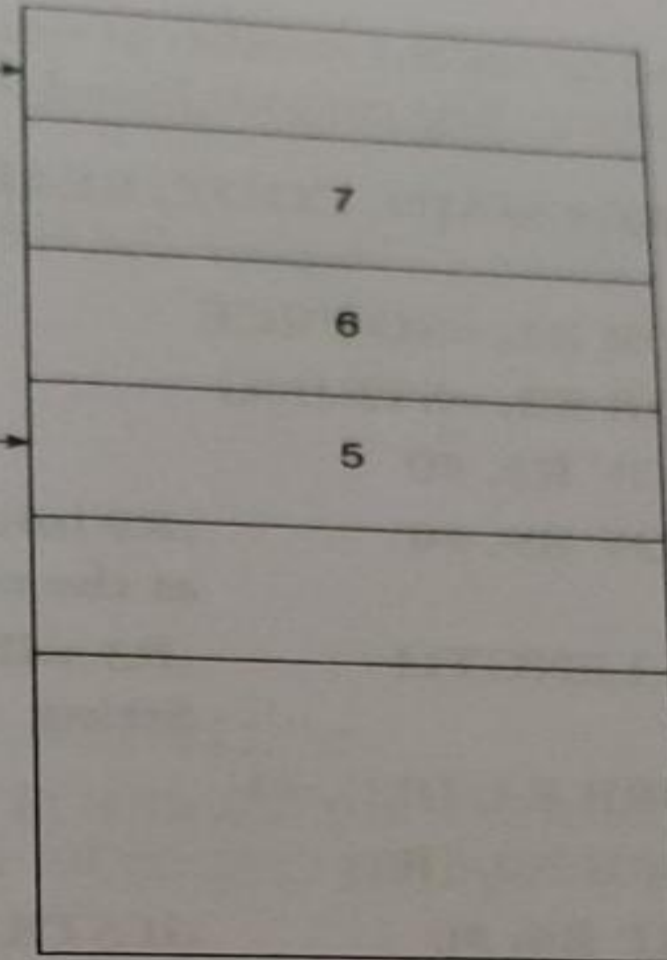
1. SP is initialised to **0x40000030**.
2. Values 5,6, and 7 are copied to registers R0, R1 and R2.
3. The content of these registers are pushed to stack.
4. The content of the highest register (R2) is stored in the highest memory location.
5. From the stack, the numbers are loaded (popped) to R9,R4 and R5.
6. Note that R9 will now have '7', which was stored in the highest memory location.
7. Figure 5.12 shows the value of SP as being 0x40000030 before the pushing operation using the STMDB instructions. After three words are pushed onto the stack, SP=0x40000018.

Before STMDB

SP = 0 x 40000030

SP = 0 x 40000018

After STMDB



5.12 Pushing into the stack